

19 Essential Interview Questions & Solutions

Source: Keerti Purswani — "20 Agentic AI System Design Interview Questions & Solutions" | Compiled by: Gemini Notebook

Q1: Design a RAG system over 10 million documents with citations under 2 seconds latency.

High-Level Architecture: Split the system into an ingestion pipeline and a query serving pipeline.

Ingestion Path:

- **Parse & Ingest:** Parse and ingest incoming documents (PDFs, text, web pages).
- **Deduplication:** Remove identical documents to save storage and prevent retrieval pollution.
- **Semantic Chunking:** Use semantic boundaries instead of arbitrary chunking, which reduces retrieval quality.
- **Metadata Tagging:** Attach metadata (tenant IDs, access permissions, versions, timestamps, source URLs) to each chunk.
- **Vector Ingestion:** Generate embeddings in batches and store them in an Approximate Nearest Neighbor (ANN) index for fast scale retrieval.

Query Serving Path:

- **Hybrid Retrieval:** Combine BM25 keyword matching (for exact terms) with dense vector retrieval (for semantic meaning) to address different search intents.
- **Reranking:** Use a stronger reranker model only on the top candidates to improve precision without blowing the latency budget.
- **Stable Citations:** Maintain stable document IDs, chunk IDs, and exact character offsets for each chunk to map every claim back to the source evidence.

Sub-2-Second Latency Optimisations:

- Apply metadata filters *before* vector retrieval to reduce the search space.
- Use ANN search instead of exact nearest neighbor search.
- Limit reranking to a small candidate set.
- Cache repeated queries and hot embeddings.

Core Trade-Off: Higher recall improves answer quality but increases retrieval latency and injects noisier context into the LLM.

Q2: Your RAG chatbot gives fluent but wrong answers. How do you debug it?

Structured Debugging Approach: Avoid blaming the LLM generation immediately or debugging randomly. Isolate the failing stage between **Retrieval** and **Generation**.

Step 1: Check Retrieval Quality

- **Ground Truth Check:** Check if the correct source passage is present in the top-K retrieved results.
- **If not present (Retrieval issue):** Inspect query rewriting, chunking strategy, embedding quality, metadata filters, and hybrid search configuration to understand why the right document was not fetched.

Step 2: Check Generation Quality

- **If present (Generation issue):** Check prompt grounding, context ordering, presence of too many distractor chunks, or whether the model is answering without enough evidence.

Systematic Evaluation (Evals): Build evaluations (evals) using real user questions to test retrieval and answer quality systematically.

Q3: Your agent keeps calling the wrong tool. How would you fix it?

Structured Trajectory Debugging: Inspect the full agent trajectory, checking the user input, the model's planning step, the tool selected, generated arguments, the tool response, and the final answer.

Common Causes: Poor tool design, vague tool descriptions, overlapping tool responsibilities, weak parameter schemas, or missing usage examples.

Mitigation Strategies:

- **Clarify Tool Boundaries:** Redefine tool boundaries so that each tool has a very specific and distinct responsibility.
- **Strengthen Schema Design:** Use required arguments, enums, and input validation before execution.
- **Few-Shot Examples:** Add few-shot examples in the prompt showing correct tool selection patterns.
- **Execution Guardrails:** For high-risk tools, add confirmation steps or policy validation layers before execution.

Key Trade-Off: Stronger constraints improve reliability but increase latency, token usage, cost, and engineering complexity.

Q4: Why can LLM training be parallelized, but inference still generates one token at a time?

Autoregressive Nature: LLMs are autoregressive models. Each generated token depends on all previously generated tokens (e.g., predicting 'Delhi' in 'The capital of India is New...' requires the preceding context).

Sequential Inference: Inference is naturally sequential because future tokens do not exist yet. The model cannot generate token 20 before generating token 1.

Parallel Training: During training, the full sentence is already known. Although the model learns next-token prediction, all positions can be processed in parallel in one batch. This enables massive parallelization on GPUs during training, whereas the inference loop remains strictly sequential.

Q5: Your company has multiple teams using LLMs. How would you standardize access?

Centralized LLM Gateway: Introduce a centralized LLM gateway as the single entry point for all teams to abstract and control direct model access.

Core Features of the Gateway:

- Authentication and team-based quotas/rate limiting.
- Model routing and fallback failovers (e.g., automatically failing over to another model if one provider goes down).
- Prompt template management, centralized logging, and cost attribution.
- Policy and guardrail checks (e.g., blocking requests that violate safety policies before they reach the model).

Core Benefits: Abstraction (teams integrate once with a stable API; the platform team swaps providers or upgrades models behind the scenes) and unified monitoring (tracking latency, error rates, costs, usage, and safety violations).

Key Trade-Off: Adds platform complexity but provides centralized governance, cost control, and stronger operational leverage.

Q6: How do multiple models (smaller vs. larger) improve an AI system instead of making it weaker?

Intelligent Allocation of Intelligence: A strong production AI system does not rely on one giant model for everything. Task complexity dictates model selection.

Smaller Models: Handle simpler, narrow tasks (e.g., intent detection, request routing, classification, information extraction, validation, guardrails, or simple draft generation) with high speed and low cost.

Larger Models: Reserved for tasks requiring deep intelligence (e.g., complex reasoning, multi-step planning, coding, or final response synthesis).

Example Workflow: A simple FAQ request is handled by a smaller model, which escalates the request to a larger model only when complexity demands it.

Key Trade-Off: Multi-model orchestration adds routing complexity, but significantly improves system latency, cost efficiency, and scalability.

Q7: Your offline evaluation score is high but users are still complaining. What might be wrong?

Metric Mismatch: The offline evaluation set does not reflect real production usage. For example:

- The evaluation set contains clean, straightforward questions, while real users ask vague, messy, and ambiguous questions.
- The evaluation set measures correctness, but users care about speed, clarity, and task completion.

Actionable Debugging:

- Analyze real failed production sessions and cluster user complaints (e.g., slow responses, repeated misunderstandings, rephrasing, or drop-offs).
- Update the offline evaluation set with these real failure patterns so that it evolves based on production reality.
- Track product-centric metrics (task completion rate, escalation rate, user correction rate, repeat question rate) alongside model-centric metrics.

Key Principle: A good model score does not equal user success. The system succeeds only when users achieve their goals.

Q8: How would you measure LLM latency for a chat application?

Latency Metrics: Latency is not a single number and must be split into two core metrics:

1. Time to First Token (TTFT):

- Measures how long the user waits before seeing the first generated token.
- Heavily impacted by prompt size during the prefill phase (larger prompts increase TTFT).

2. Tokens Per Second (TPS):

- Measures how fast the response streams once generation has started.
- Influenced by model size and GPU load during the decoding phase.

Product Impact: In chat products, TTFT is psychologically critical because starting the streaming response quickly makes the system feel faster, even if the total response takes time.

Q9: Your RAG works for 10,000 documents but fails at 100 million. What changes?

Distributed Systems Shift: Scaling RAG from 10k to 100 million documents makes it a distributed systems problem, as a single index on one machine is no longer practical.

System Changes:

- **Sharding & Distributed Retrieval:** Split the index into shards. Collect, merge, and globally rerank partial results from multiple shards, introducing coordination overhead and tail latency.
- **Early Metadata Filtering:** If tenant isolation or permission constraints exist, filtering after retrieval is wasteful. Apply filtering early to reduce the search space.
- **Ingestion Overhead:** Full reindexing is impractical. Shift to incremental indexing, versioning, freshness handling, and aggressive caching for hot queries.

Key Trade-Off: Better recall at scale comes with higher latency, increased infrastructure cost, and system complexity.

Q10: Design memory for an agentic application.

Four-Layer Memory Architecture:

1. **Short-Term Memory:** The active working context for the current task (objectives, recent tool outputs, reasoning, pending steps). Typically lives inside the LLM context window.
2. **Long-Term Memory:** Persistent knowledge that survives beyond a single interaction (user preferences, learned facts, historical decisions). Stored externally and retrieved when relevant.
3. **Shared Memory:** The coordination layer in multi-agent systems, providing agents with shared visibility into task progress and intermediate outputs (e.g., Researcher writes to shared memory for Writer agent).
4. **Episodic Memory:** A timestamped log of past execution episodes that the system can analyze and learn from over time.

Key Trade-Off: Better memory improves context quality but increases storage requirements, retrieval complexity, and consistency challenges.

Q11: Your retrieved documents contain prompt injections. How do you defend the system?

Untrusted Data Principle: Treat retrieved documents strictly as untrusted data, never as system instructions. Do not let documents override system guidelines.

Defense-in-Depth Architecture:

- **System Prompt Separation:** Explicitly separate developer instructions from retrieved document content in the prompt.
- **No Direct Actions:** Retrieved content should never directly trigger actions or execution. Validate every tool call server-side.
- **Least Privilege Access:** Restrict agent access rights (e.g., read-only access if write or delete is not strictly necessary).
- **Prompt Injection Detection:** Scan retrieved content with dedicated detection layers for suspicious instruction patterns (used as an extra layer, not primary defense).
- **Human-in-the-Loop (HITL):** Require explicit human confirmation for sensitive actions like payments, deletions, or external API calls.

Key Principle: Real security must be enforced outside the prompt boundary through application-level constraints.

Q12: Why is embedding-based semantic search not enough for a production RAG system?

Semantic Search Strength: Capturing meaning and intent (e.g., mapping 'how do I reset my password' to 'steps to recover account access').

Semantic Search Weakness: Fails on exact, token-specific lookups like error codes, API names, config keys, function names, or product IDs (e.g., searching 'error 429' results in too many unrelated matches).

Production Hybrid Retrieval:

- Combine semantic search (for intent) with keyword-based search like BM25 (using an inverted index mapping terms to documents for fast, exact lookups).
- Merge results and rerank the top candidates.

Key Trade-Off: Semantic search improves recall for meaning-based queries, while keyword/exact retrieval is critical for precision on technical or token-specific queries.

Q13: In RAG, should you optimize for recall or for precision?

Definitions: Recall measures if we fetched all relevant documents; precision measures what proportion of retrieved documents are actually relevant.

Recall Bottleneck: Maximizing recall blindly floods the LLM context window with noisy or irrelevant chunks. This confuses the model, increases latency/cost, and degrades answer quality.

Precision Bottleneck: Optimizing too aggressively for precision can lead to missing critical evidence entirely, which is catastrophic.

Production Pattern: Optimize retrieval for reasonably high recall first, and then improve precision using a stronger reranking model before sending the context to the LLM.

Key Trade-Off: Higher recall improves coverage, while higher precision improves answer quality and reduces noise. Production RAG requires a balanced approach.

Q14: What does multi-tenant isolation actually mean in AI systems?

Core Concept: Multiple customers share the same platform, but one tenant must never see, influence, or interfere with another tenant's data or workload.

Multi-Layer Isolation:

- **Data Isolation:** Embeddings, documents, memory, and history of Tenant A must be inaccessible to Tenant B. Enforce tenant-aware indexing and filtering during retrieval.
- **Access Isolation:** Enforce server-side authorization based on verified tenant identity, never trusted from prompt inputs.
- **Compute Isolation:** Enforce quotas, rate limits, and workload controls per tenant to prevent one 'noisy tenant' from consuming all capacity and degrading latency for others.
- **Cache & Tool Isolation:** Prevent cached responses from leaking across tenants. Ensure agents for one tenant cannot access another tenant's APIs, files, or actions.

Key Trade-Off: Stronger isolation improves security and reliability but increases complexity, cost, and operational overhead.

Q15: When do we need an agentic RAG?

Standard RAG: Single-step retrieve-and-generate process. Highly effective for simple, direct factual questions.

Agentic RAG (Reasoning-Driven Retrieval): Required when a task is multi-step, requires decomposition, or depends on intermediate findings.

Example: Answering 'When can I receive the replacement for my order?' requires retrieving order details, product details, and shipping timelines across separate systems, synthesizing the information dynamically.

Capabilities: The system can plan, use tools, retrieve from multiple sources iteratively, and self-correct.

Key Trade-Off: Agentic RAG provides vastly higher capability but introduces higher latency, costs, orchestration complexity, and more potential failure nodes.

Q16: How would you design an AI agent that can recover from failures?

1. **Classify Failures:** Isolate temporary failures (e.g., API timeouts) from permanent failures (e.g., invalid inputs) as they require different recovery paths.
2. **Temporary Failures:** Apply bounded retries with exponential back-off.
3. **Permanent Failures:** Do not retry; route the workflow to an alternative path.
4. **Resumability:** Persist agent state and checkpoints. If a multi-step workflow fails mid-way, allow it to resume from the last successful checkpoint instead of restarting from scratch.
5. **Built-in Fallbacks:** If a primary tool fails, try an alternative tool. If automated recovery fails, escalate to human-in-the-loop (HITL) handoff.

Key Trade-Off: Better recovery improves system reliability but introduces substantial state management, checkpointing overhead, and workflow complexity.

Q17: How would you handle conflicting documents in a RAG system?

Avoid Guessing: Do not allow the model to guess or pick a source randomly when two retrieved documents disagree (e.g., remote policy says 3 days vs. 2 days).

Metadata-Driven Conflict Resolution: Apply deterministic rules using document metadata (such as version numbers, timestamps, or source authority). For example, prefer the latest version or the higher-authority source.

Fallback Clarification: If conflict resolution rules cannot resolve the disagreement automatically, instruct the model to explicitly state the disagreement rather than making an assumption.

Key Trade-Off: Reliable answers require strict document governance and rich, well-maintained metadata.

Q18: Your AI traffic grows 100x. How do you scale inference?

Identify Bottlenecks: Pinpoint if the bottleneck lies in the model, GPUs, request rate, or latency requirements.

Inference Scaling Palette:

- **Caching:** Cache identical queries, retrieval results, or prompt prefixes to bypass redundant model calls.
- **Batching:** Group incoming requests arriving around the same time and process them in a single parallel inference pass to maximize GPU utilization.
- **Replicas:** Spin up multiple model instances behind a load balancer to serve requests in parallel.
- **Queuing:** Queue incoming requests during traffic spikes to protect the system from dropping requests, processing them as capacity allows.

Key Trade-Offs: Caching and batching can increase latency for individual requests while boosting throughput. Replicas increase costs significantly. Queuing protects the system but adds wait times.

Q19: How do you prevent agent memory from becoming stale or incorrect?

Memory is Not Truth: Treat stored memory as a source of context, not absolute truth. Facts, user preferences, and tasks change over time.

Mitigation Strategies:

- **Metadata Tagging:** Attach metadata like last-updated timestamps and source confidence levels to evaluate freshness.
- **Validation:** Validate critical memories against external sources of truth before using them in reasoning.
- **Forgetting/Pruning:** Implement a mechanism to expire, archive, or remove old, unused, low-confidence, or contradictory memories to maintain quality.

Key Principle: Memory provides context, not truth. Without validation, freshness checks, and forgetting, memory quality degrades over time.